PRODUCTION ARCHITECTURE

How the TN EC system would actually be built

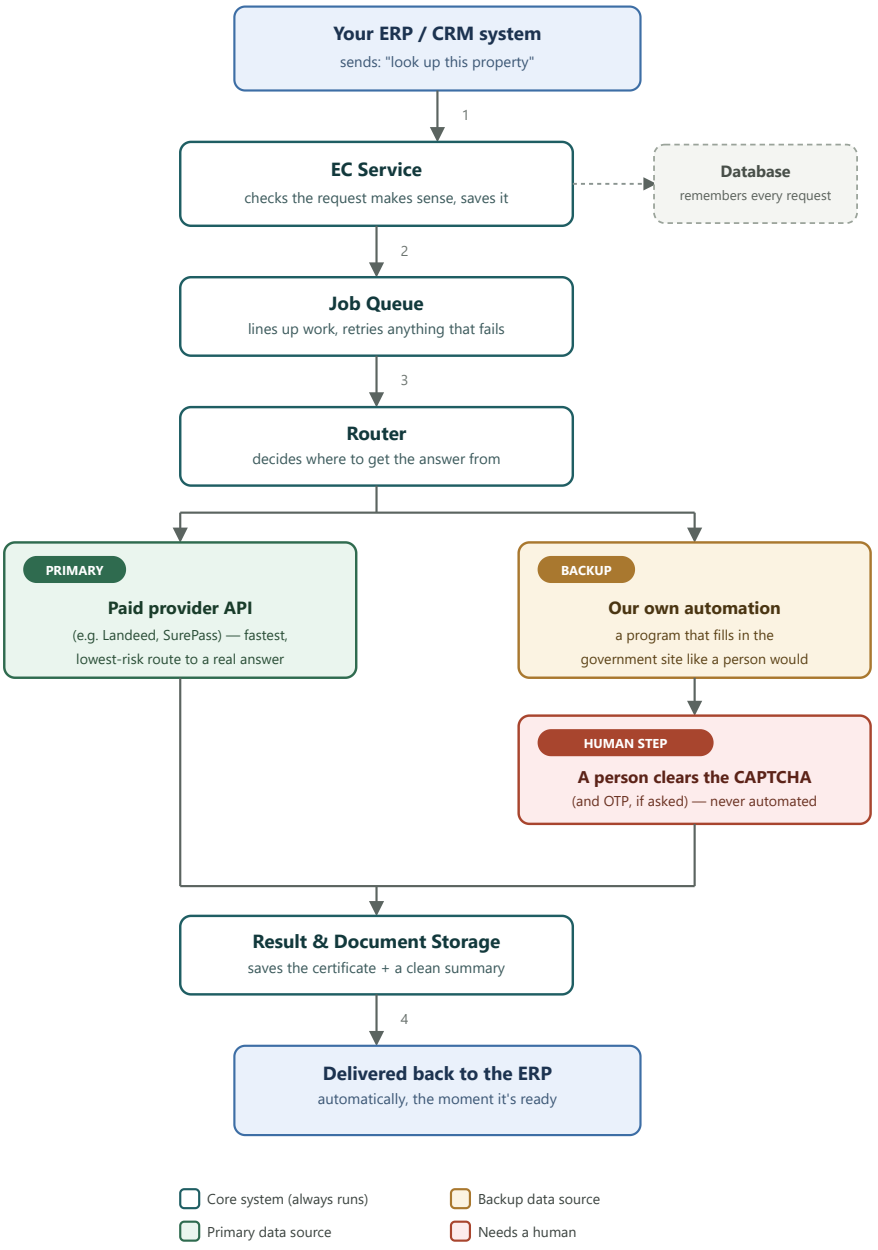
A visual walkthrough of the design: how a request flows through the system, what's essential versus what can wait, and which tools it's built from.

Prepared by: Raakesh R

Date: 11 September 2026

How a request flows through the system

One property lookup, from the moment the ERP asks, to the moment it gets an answer back — automatically, except for one clearly marked step.



What's essential versus what can wait

Not every piece needs to exist on day one. Here's the split between what the system cannot work without, and what makes it better later.

Core — needed from day one

- **EC Service** — the front door the ERP talks to
- **Job Queue** — so a slow lookup never blocks the next one
- **Router** — picks a data source, switches automatically if one fails
- **Database** — a record of every request, for audit and support
- **At least one working data source** — the paid provider, to start
- **A way to get the answer back to the ERP** — automatically

Adds value later — not blocking

- **Our own backup automation** — for when the paid provider can't cover a case
- **A second paid provider** — for price leverage and redundancy
- **Reading old, scanned certificates** — pre-1987 records need extra handling
- **Remembering recent identical lookups** — avoids paying twice for the same answer
- **Usage dashboards & alerts** — nice for operations, not required to function
- **Paying for an official signed copy automatically** — can start as a manual step

> **The one thing that never moves to "later":** a real person always clears the CAPTCHA step, and always double-checks a "nothing found" result before it's treated as good news. That's true on day one and true at any scale.

Tools this would be built with

Plain-English purpose for each, grouped by what job it does — not a vendor pitch, just what each piece is for.

TOOL	CATEGORY	WHAT IT'S FOR, IN PLAIN TERMS
Node.js + TypeScript	Language	The programming language the whole service is written in — TypeScript catches mistakes before they reach real users.
Fastify	Web framework	Handles the "front door" — receiving a request from the ERP and sending a reply.
PostgreSQL	Database	Stores every request, its status, and a full history — the record-keeper.
A job queue (e.g. BullMQ / Redis)	Task management	Makes sure lookups happen in order, retry automatically if something fails, and one slow request doesn't hold up the rest.
Playwright	Browser automation	Drives a real web browser automatically — used only for the backup route, filling in the government site the way a person would.
An OCR tool (e.g. Tesseract)	Text recognition	Reads text out of old, scanned certificate images so they can be turned into structured data too.
A signed-webhook system	Delivery	The mechanism that reliably sends the finished answer back to the ERP, and proves it really came from us.
Encrypted file storage	Documents	Where the actual certificate files and summaries are kept, safely and privately.

None of this is exotic — every tool above is a mainstream, widely-used choice, picked because it's boring and reliable, not because it's novel. The full technical report has the detailed reasoning for each choice.